DFAs and NFAs

A DFA (Deterministic Finite-state Automaton) is a simple machine that can solve decision problems.

Defined as a 5-tuple $(Q, \Sigma, \delta, q_0, F)$:

- Q is a finite set of states
- Σ is a finite alphabet
- δ is the transition function
- q_0 is the start state, $q_0 \in Q$
- F is the set of accepting states, $F \subseteq Q$

A DFA accepts a word $w = w_1 w_2 \dots w_n$ if there is a sequence of states beginning at q_0, following the transition function at each step i for w_i, and ending in some accepting state $q_n \in F$.

A language is regular if some DFA recognises it.

Regular languages L are defined using regular expressions.

Operations: All these ops are closed, i.e. output a regular language.

Union: $L_1 \cup L_2 = \{w \mid w \in L_1 \text{ or } w \in L_2\}$

Given L_1 recognised by $DFA_1 : (Q_1, \Sigma, \delta_1, q_1, F_1)$, L_2 recognised by $DFA_2 : (Q_2, \Sigma, \delta_2, q_2, F_2)$

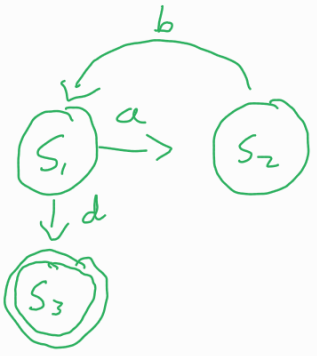
The DFA representing the union can be created using pairs of states from DFA_1 and DFA_2. This new DFA $DFA_3 = (Q, \Sigma, \delta, q_0, F)$ is defined as:

$Q = Q_1 \times Q_2$, $\delta((s,u), a) = (\delta_1(s,a), \delta_2(u,a)) \quad \forall s \in Q_1, u \in Q_2, a \in \Sigma$, $q_0 = (q_1, q_2)$,

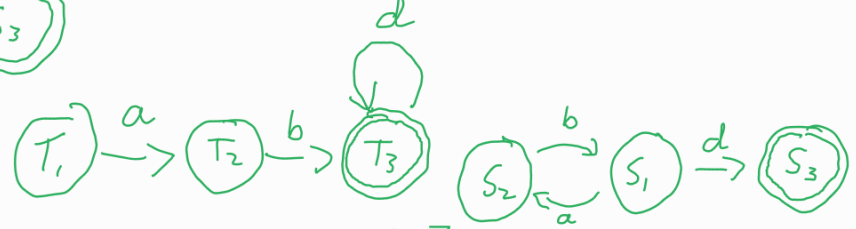
$F = \{(s,u) \in Q \mid s \in F_1 \vee u \in F_2\}$

Example:

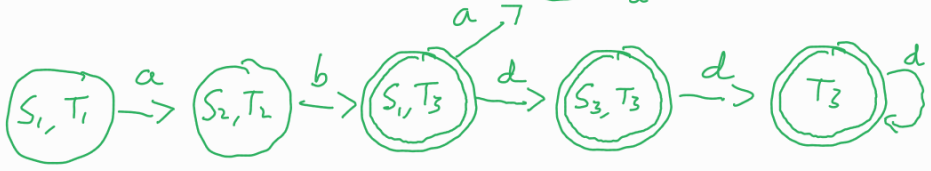
L_1: $(ab)^*d$ $\quad DFA_1$:



L_2: abd^* $\quad DFA_2$:



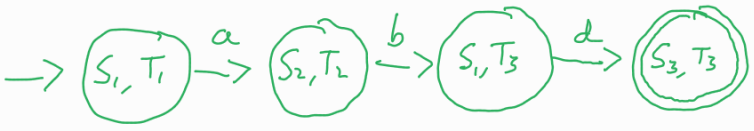
$L_1 \cup L_2$ $\quad DFA_3$:



Intersection : $L_1 \cap L_2 = \{w \mid w \in L_1 \wedge w \in L_2\}$

As Union, but $F = \{(s,u) \in Q \mid s \in F_1 \wedge u \in F_2\}$

Using above example, $L_1 \cap L_2$ would be represented by:



(the word 'abd' is the only word accepted by both L_1 and L_2)

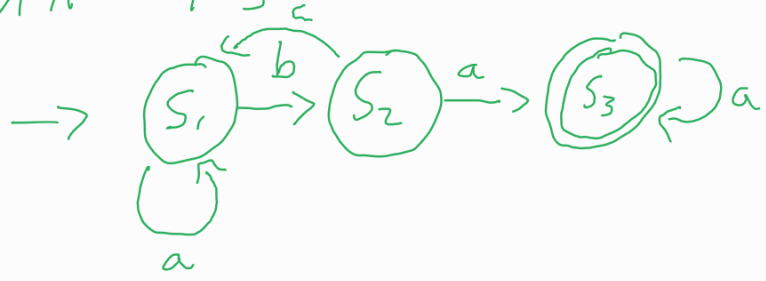
Complement : $\bar{L} = \Sigma^* \setminus L$

This is the set of all possible words that can be made using letters in Σ except those accepted by L. The DFA that accepts $\bar{L}$ can be obtained from the DFA accepting L by swapping the accepting/non-accepting states.

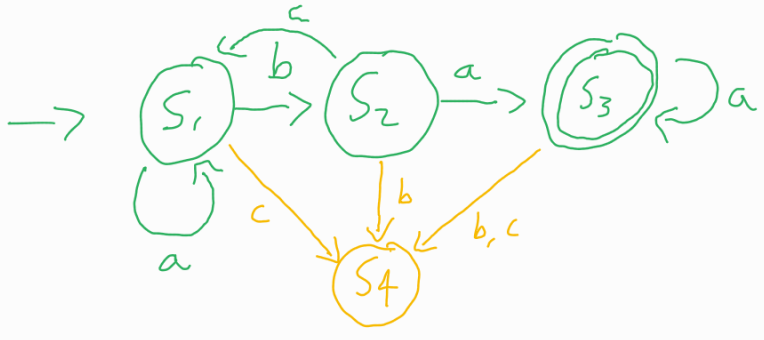
I.e. if $L = (Q, \Sigma, \delta, q_0, F)$, then $\bar{L} = (Q, \Sigma, \delta, q_0, \{q \in Q \mid q \notin F\})$

Example : $\Sigma = \{a, b, c\}$

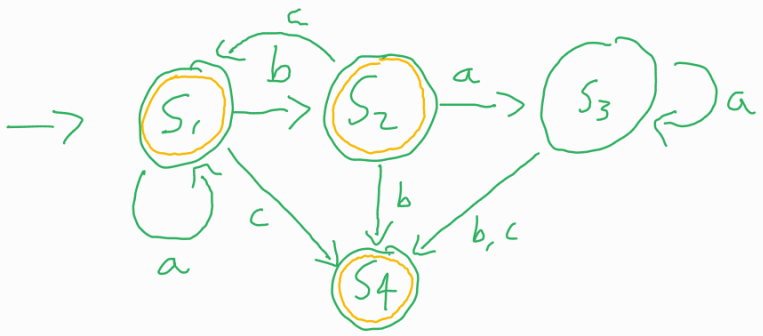
DFA accepting L is:



First, Complete the DFA by ensuring every possible transition is defined:



Now, Swap accepting/non-accepting states:



This DFA recognises $\bar{L}$.

Difference: $L_1 \setminus L_2 = \{w \mid w \in L_1 \wedge w \notin L_2\}$

This is defined as $L_1 \cap \bar{L_2}$ so just combine those two operations !!

Star: $L^* = \{w_1 w_2 \ldots w_k \mid k \geq 0 \wedge w_i \in L \; \forall \; 1 \leq i \leq k\}$

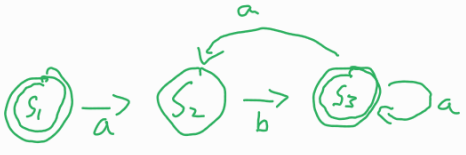
To convert a DFA recognising L to one recognising L^*:

- make the start state accepting
- wrap around.

However this isn't possible with DFAs. Consider:

DFA recognising L: 

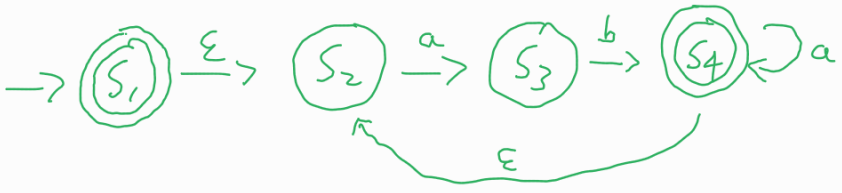
Once we are at S_3 we have no way of knowing whether an a is a continuation of the current w_i or the start of the next. We need:

FA recognising L^*: 

which has two outbound a transitions at S_3. As a result of this, this FA is an NFA (Non-deterministic Finite-state Automaton). An NFA is a 5-tuple as with DFAs, however empty transitions ε are now allowed, as are multiple transitions for the same symbol.

An NFA accepts if there is a possible path through the machine that ends on an accepting state.

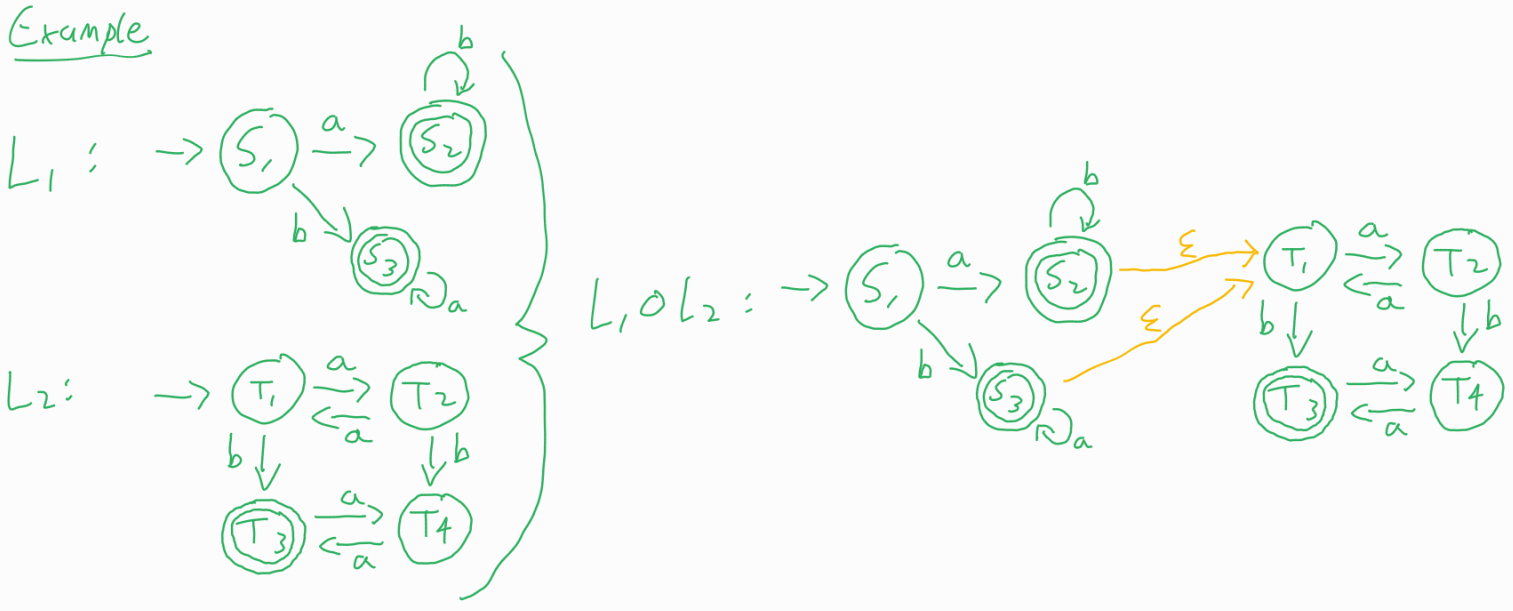
We can also write the NFA recognising L^* using ε-transitions:



Concatenation : $L_1 \circ L_2 = \{ w_1 w_2 \mid w_1 \in L_1 \wedge w_2 \in L_2 \}$

With NFAs, concatenation is trivial : just add an ε-transition from every accepting state of the DFA for L_1 to the start state of the DFA for L_2

Example

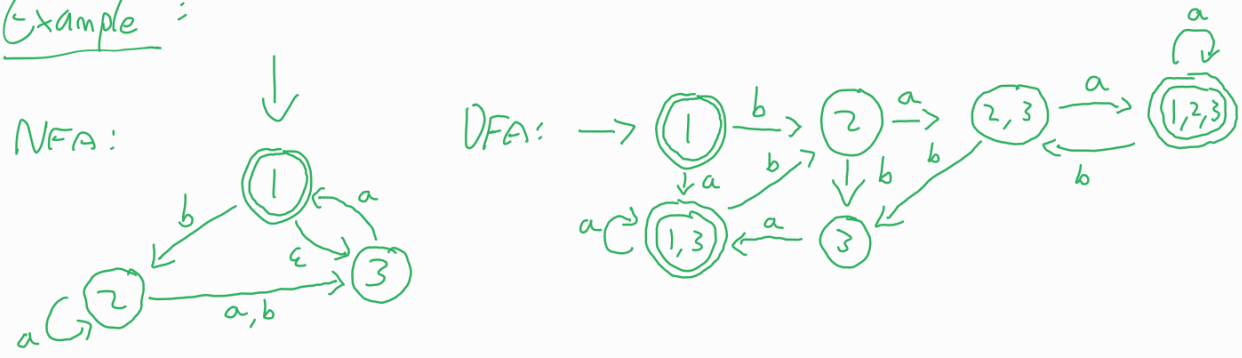


NFA to DFA

Every NFA has an equivalent DFA.

Each state in the DFA corresponds to a subset of states in the NFA. If one member of this subset is an accepting state in the NFA, then the state in the DFA is accepting.

Example :



$\delta(1, a) = 1, 3$ (due to ε-transition)

$\therefore$ on DFA (1) $\xrightarrow{a}$ (1,3)

repeat for all transitions.

Generalised NFA

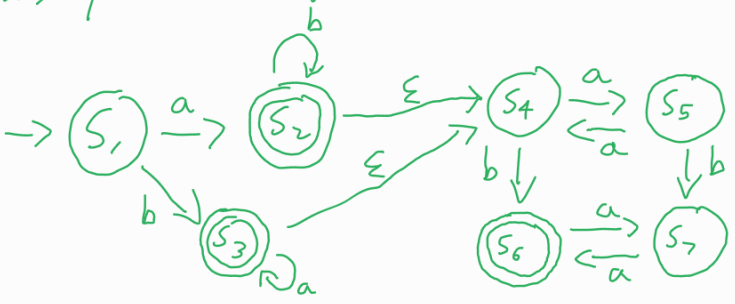
A generalised NFA is one with:

- One start state with no incoming edges
- One accept state with no outgoing edges
- Every transition labelled with a regular expression.

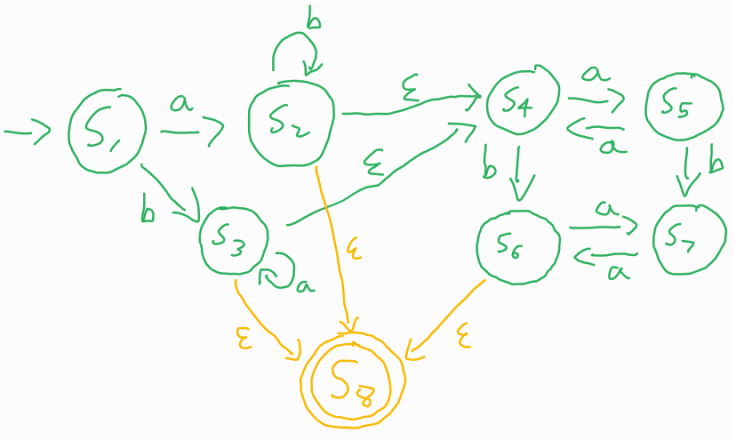
Any DFA/NFA can be converted to a gNFA by adding arbitrary start and accept states with empty transitions.

A gNFA can be converted into a regular expression by removing internal states.

Example: find the regular expression represented by:

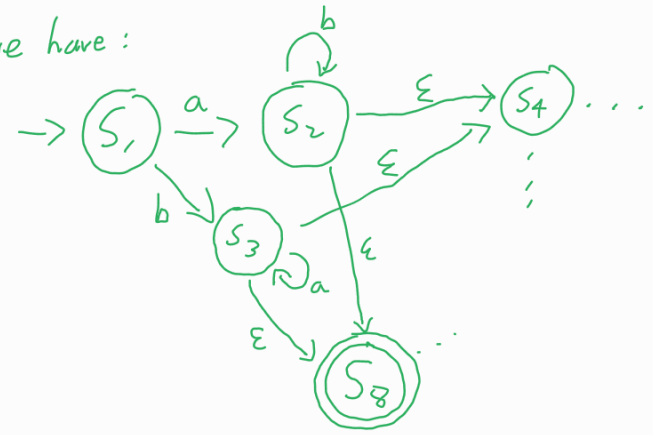


① Convert to gNFA.
Need one accepting state.

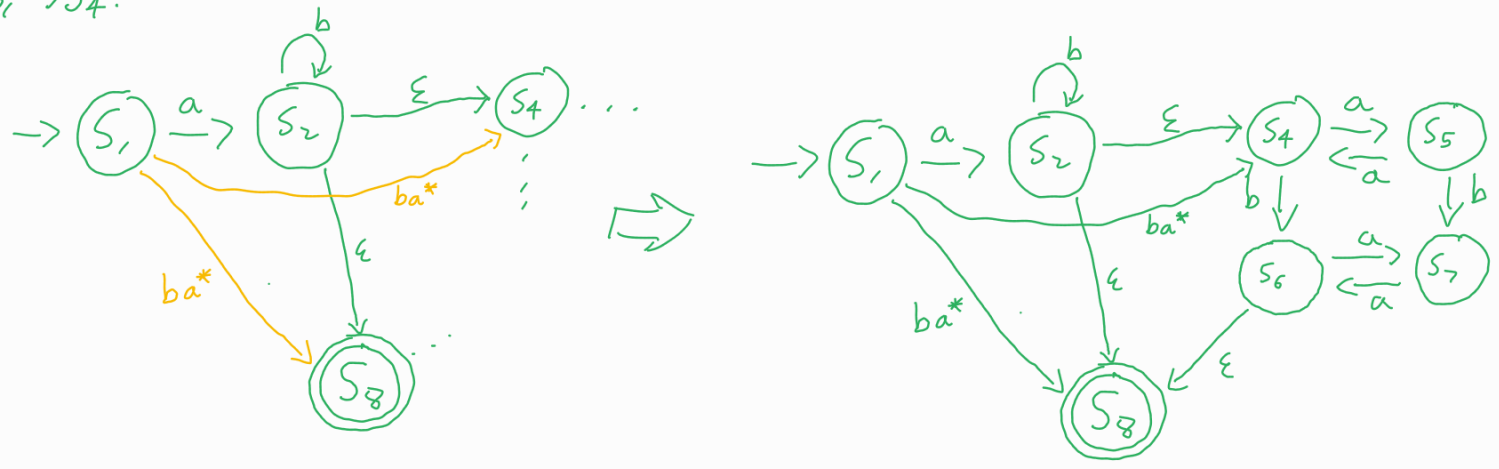


② Eliminate states one by one

We have:

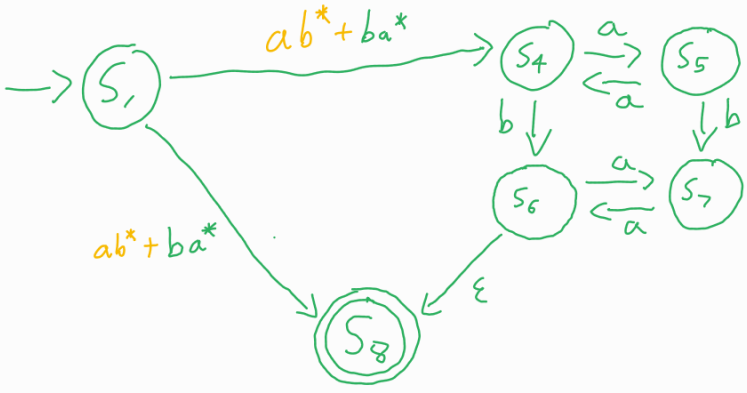


We can eliminate S_3 by rewriting the transitions between $S_1 \to S_8$ and $S_1 \to S_4$:

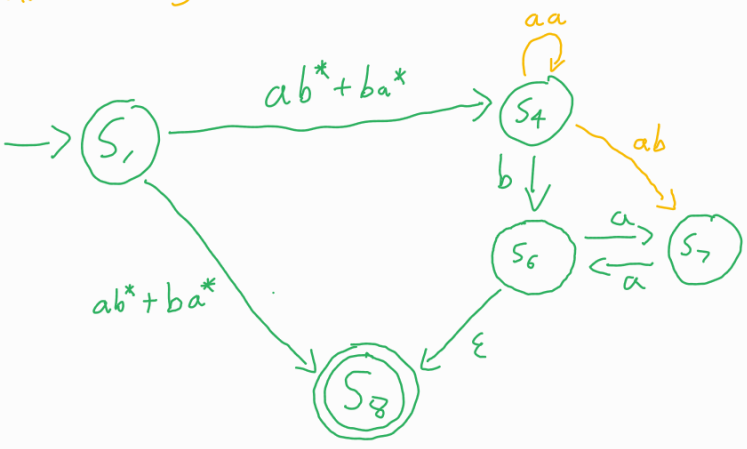


This can be repeated for every state in the gNFA:

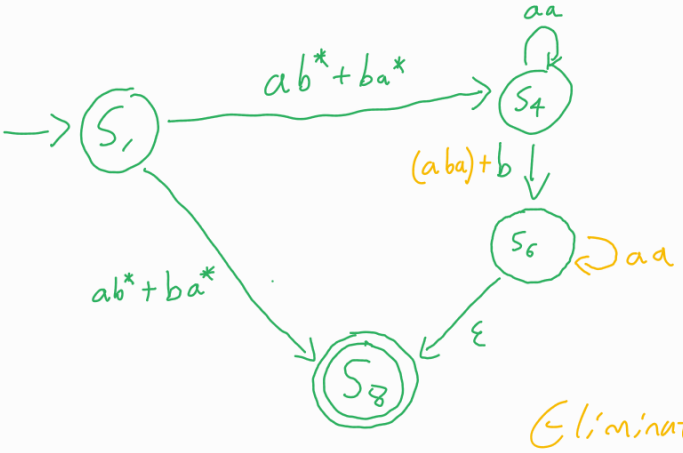
Eliminate S_2:



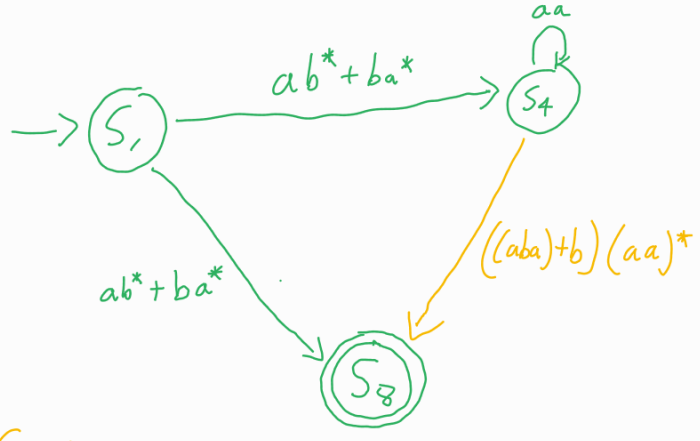
Eliminate S_5:



Eliminate S_7:



Eliminate S_6:



Eliminate S_4:

$$(ab^*+ba^*)+(ab^*+ba^*)(aa)^*((aba)+b)(aa)^*$$

Non-regular languages

A language is non-regular if there does not exist a DFA/NFA that recognises it.
The Pumping Lemma can be used to prove that a language is not regular:

Pumping Lemma: For every regular language L there exists a 'pumping length' p such that all words $w \in L$ of length $\geq p$ can be divided into three parts:

$w = xyz$

Such that

- $xy^iz \in L \quad \forall i \in \mathbb{N}$
  - $xz \in L$ (case when $i = 0$)
- y is non-empty
- the length of $xy \leq p$

Example: $L = \{0^n 1^n \mid n \in \mathbb{N}\}$

Assume it is regular → follows the Pumping Lemma

Consider $0^p 1^p \in L$, where p is the pumping length.

Since $|0^p 1^p| \geq p$, then we should have $0^p 1^p = xyz$ as defined above.

Since $|xy| \leq p$, then $x = 0^i$, $y = 0^j$ and $z = 0^{p-i-j} 1^p$ for some i, j s.t. $i+j \leq p$, $i \geq 0, j > 0$

By the Pumping Lemma, $xy^2z \in L$, however $xy^2z = 0^i 0^{2j} 0^{p-i-j} 1^p = 0^{p+j} 1^p \notin L$.

This is a contradiction $\Rightarrow L$ is not regular.

DFA Minimisation

Minimisation is finding a DFA that recognises the same language in as few states as possible. This is achieved by:

- removing all states that are not reachable from the start state
- Contracting a set of equivalent states into a single state.

Two states s and t are equivalent if:

$$\{w \in \Sigma^* \mid \hat{\delta}(s, w) \in F\} = \{w \in \Sigma^* \mid \hat{\delta}(t, w) \in F\}$$

i.e. the set of input strings starting at s that are accepted by the DFA is the same as the set of input strings starting at t that are accepted by the DFA.

Note $\hat{\delta}(q, w)$ is the extended transition function, outputting the state in the DFA that we end up in after trying the input string w from state q.

If there exists some word u such that:

$\hat{\delta}(s,u) \in F$, $\hat{\delta}(t,u) \notin F$

or

$\hat{\delta}(s,u) \notin F$, $\hat{\delta}(t,u) \in F$

then states s and t are not equivalent.

The team splitting algorithm can be used to minimise a DFA.

We can also use the minimisation algorithm to show that two DFAs are isomorphic.